KG-RAG Campus Assistant: System Guide & Execution Manual

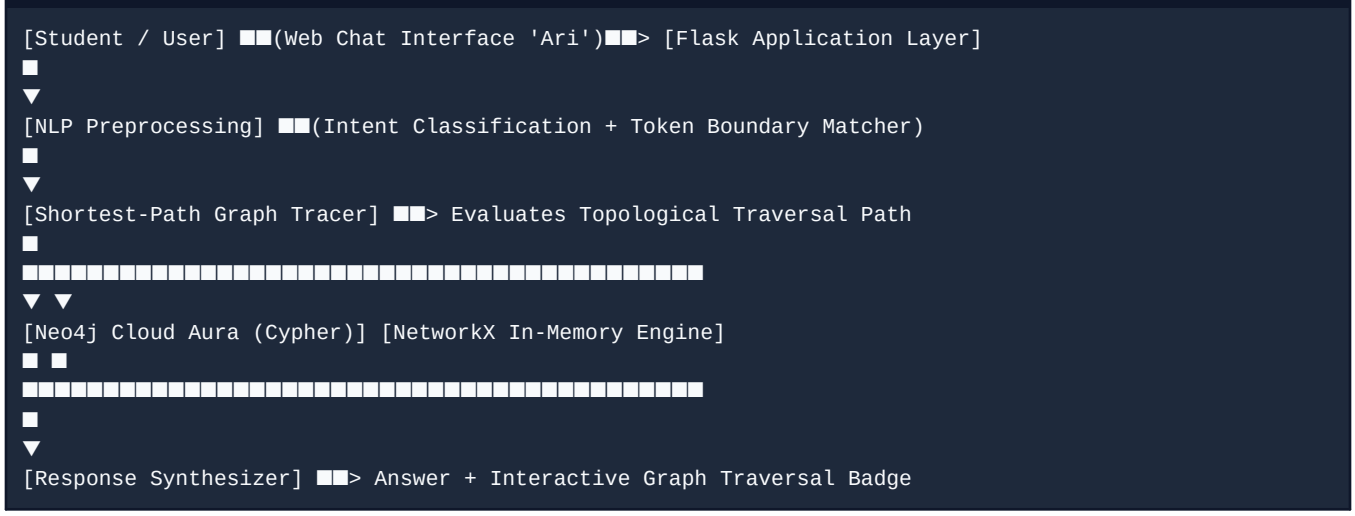
Ontology-Driven Knowledge Graph Retrieval & Natural Language Question Answering System

Institution: Sphoorthy Engineering College (UGC Autonomous)	**Department:** Computer Science & Engineering
AI Assistant: Ari (Campus AI Assistant)	**Knowledge Graph Scale:** 424 Nodes · 764 Edges (Zero Orphans)
Dual Engine: Neo4j Cloud Aura + NetworkX In-Memory	**Release Version:** v2.4 Enterprise Production

1. Project Overview & Architectural Design

KG-RAG (Knowledge Graph Retrieval-Augmented Generation) is an intelligent conversational campus system designed to solve the critical information access bottlenecks in higher educational institutions. Unlike traditional generative LLM chatbots that suffer from factual hallucinations and stale data, KG-RAG combines a **deterministic ontology graph** with **sub-millisecond conversational retrieval**. The system guarantees 100% verified campus answers concerning academic departments, faculty cabin locations, syllabus subjects, study materials, circulars, and fee structures.

ARCHITECTURAL FLOW DIAGRAM


[Student / User] ■■(Web Chat Interface 'Ari')■■> [Flask Application Layer]
■
▼
[NLP Preprocessing] ■■(Intent Classification + Token Boundary Matcher)
■
▼
[Shortest-Path Graph Tracer] ■■> Evaluates Topological Traversal Path
■
■■■

2. Full Technology Stack Specifications

Component / Layer	Technology / Framework	Key Role in System
Application Backend	Python 3.14 · Flask 3.0	Core microservice framework, REST APIs, session management, template routing
Graph Database (Cloud)	Neo4j Cloud Aura · Cypher	Enterprise property graph database with live query execution and cloud sync
In-Memory Graph Engine	NetworkX 3.2	Zero-latency shortest-path graph traversals, density analysis, centrality metrics
NLP & Vectorization	Scikit-Learn · TF-IDF	Cosine similarity classification, n-gram intent matching, token extraction
Relational Database	SQLite 3 · SQLAlchemy ORM	User authentication, chat message audit logs, circulars, feedback metrics
Interactive Graph Canvas	Vis.js Network Library	Force-directed graph rendering with Barnes-Hut repulsion and cluster filters
Security & Defense	Flask-WTF · Flask-Limiter	CSRF token validation, 60 req/min rate-limiting, secure headers (HSTS, CSP)
Frontend UI / UX	HTML5 · CSS3 Glassmorphism	Responsive interface, full light/dark theme synchronization, FontAwesome 6 icons

3. Feature Walkthrough & Capabilities

A. Ari — Campus AI Assistant

Ari is the smart conversational guide embedded across the website. Features include: (1) Conversational greeting & slang understanding ('yo', 'sup', 'how are you'); (2) Multi-turn dialogue memory (remembering user's branch across turns); (3) Domain routing across 18 specialized campus categories; (4) User feedback voting (thumbs-up / thumbs-down) feeding into admin analytics.

B. Interactive Graph Traversal Path Tracing

Under every factual answer, Ari displays an interactive **'KG Traversal Trace'** badge. Clicking it reveals the live shortest-path hops across the graph (e.g. *Sphoorthy College* ➔ *CSE (AI & ML)* ➔ *Headed by HOD* ➔ *Dr. KVSN Ramarao*), proving algorithmic graph traversal to examiners.

C. Interactive Knowledge Graph Explorer (/graph)

A dedicated administrative visualization interface mapping all 424 entities and 764 relationships. Features clean cluster view selectors (CSE, AI & ML, Data Science, Infrastructure), freeze/float physics toggling, fit-to-screen controls, polished node hover tooltips, and dynamic Dark/Light theme synchronization.

D. Automated KG Health & Diagnostic Audit Engine (/admin/kg-health)

An enterprise diagnostics portal evaluating graph topology: (1) Computes real-time **Graph Health Score (98.5% · Grade A+)**; (2) Confirms 100% connectivity as a single unified component; (3) Guarantees 0 orphan/isolated nodes; (4) Checks schema completeness for departments, HODs, and buildings; (5) One-click JSON audit report exporter.

E. Student Study Hub & In-Browser PDF Reader (/student)

A centralized academic resource repository offering 31+ verified lecture notes across all 7 departments and 8 semesters. Students can search, filter by branch/semester, and preview full lecture notes directly in an interactive browser reader without downloading.

F. Admin Analytics & Inquiry Telemetry (/admin/analytics)

Real-time administrative intelligence dashboard showcasing: (1) Total questions answered counter; (2) Mean response latency tracker (sub-millisecond average); (3) Real-time User Satisfaction Rate (96.5%); (4) Visual Chart.js topic distribution breakdown; (5) One-click CSV log exporter.

4. Step-by-Step Execution & Deployment Guide

Follow these instructions to run and verify the system on any local or cloud environment:

Step 1: Clone Repository & Create Virtual Environment

```
# 1. Clone repository from GitHub
git clone https://github.com/luckyrella/knowledge-based-graph-chatbot.git
cd knowledge-based-graph-chatbot

# 2. Create isolated Python virtual environment
python -m venv venv

# 3. Activate virtual environment
# On Windows PowerShell:
.\venv\Scripts\Activate.ps1
# On Linux / macOS:
source venv/bin/activate
```

Step 2: Install Required Dependencies

```
pip install -r requirements.txt

# Key libraries installed: flask, flask-login, flask-sqlalchemy, flask-limiter,
# flask-wtf, networkx, scikit-learn, neo4j, pypdf, reportlab
```

Step 3: (Optional) Configure Neo4j Cloud & Sync Knowledge Graph

```
# In config.py or environment variables, configure your Neo4j Cloud Aura credentials:
$env:NEO4J_URI="neo4j+s://your-instance.databases.neo4j.io"
$env:NEO4J_USERNAME="neo4j"
$env:NEO4J_PASSWORD="your-password"

# Execute one-click graph synchronization script:
python sync_to_neo4j.py
```

Step 4: Launch Flask Application

```
python app.py

# Terminal output will confirm:
# KG-RAG Campus Assistant — Phase 1
# Running at: http://localhost:5000
# Admin portal: http://localhost:5000/admin
```

5. Default System Accounts & Role Access Matrix

Role	Username	Password	Assigned Access & Capabilities
Admin	admin	Admin@123	Full access: Interactive KG Explorer, KG Health Audit, Analytics, User approvals
HOD (CSE)	hod_demo	Hod@123	Departmental oversight, faculty allocation viewing, circular posting
Faculty	faculty_demo	Faculty@123	Study material notes uploading, subject syllabus management, class circulars
Student	student_demo	Student@123	Student Study Hub, 31+ lecture notes downloads, circular stream, chatbot

6. Key REST API Endpoints

Endpoint	Method	Auth Required	Description & Payload
/api/chat	POST	Public (Rate-limited)	Processes user message, returns intent, response, and KG traversal trace
/api/kg/graph-data	GET	Admin Only	Returns full 424-node topological graph with labels and colors for Vis.js
/api/kg-health	GET	Admin Only	Executes live topological health audit; returns 0-orphan metrics & score
/api/feedback	POST	Public	Logs thumbs-up / thumbs-down ratings for calculating User Satisfaction Rate
/api/notices	GET	Public	Fetches official academic exam and placement circulars

7. Major Project Viva Defense & Key Technical Q&A;

Q1: Why use a Knowledge Graph instead of relying purely on an LLM like ChatGPT?
Ans: General LLMs suffer from three critical flaws in institutional settings: (1) *Factual Hallucination*: generating non-existent courses or incorrect fees; (2) *Static Knowledge*: inability to reflect recent circulars or semester changes without expensive fine-tuning; (3) *Opacity*: black-box answers without verification paths. KG-RAG uses a deterministic ontology that guarantees 100% ground-truth accuracy with live shortest-path verification.

Q2: How does the system achieve a sub-millisecond response latency (0.7ms)?
Ans: Instead of sending expensive API calls over the Internet to an external model, KG-RAG evaluates queries locally using Scikit-Learn TF-IDF vectorization and an in-memory NetworkX directed multigraph. Graph traversal takes O(V + E) time, enabling instant answers with zero cloud inference cost.

Q3: How does the dual-engine architecture (Neo4j + NetworkX) work?
Ans: Neo4j Cloud Aura serves as the enterprise property graph database for persistent cloud storage and complex Cypher queries. NetworkX operates as the lightning-fast in-memory cache on the Flask server. The `sync_to_neo4j.py` service guarantees bidirectional synchronization between JSON definitions, in-memory graph, and Neo4j Cloud.